



PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

Campo	Información
Denominación del Programa de Formación	ANÁLISIS Y DESARROLLO DE SOFTWARE
Código del Programa de Formación	228118
Nombre del Proyecto Formativo (si aplica)	Desarrollo de una solución de software para una necesidad del contexto productivo
Fase del Proyecto (si aplica)	Planeación
Actividad de Proyecto Formativo (si aplica)	Diseñar y documentar la arquitectura de la solución de software
Competencia	220501095 – Diseñar y desarrollar soluciones de software de acuerdo con el análisis de requisitos y las buenas prácticas de ingeniería de software
Resultado de Aprendizaje	Desarrollar la arquitectura de software de acuerdo al patrón de diseño seleccionado.
Actividad de Aprendizaje	Diseño y documentación de la arquitectura de software
Evidencia de referencia	Desarrollar la arquitectura de software de acuerdo al patrón de diseño seleccionado
Duración de la guía	80 horas

2. PRESENTACIÓN

La arquitectura de software constituye una decisión fundamental en el desarrollo de soluciones tecnológicas, porque permite organizar los componentes del sistema, establecer sus responsabilidades y definir la manera en que se comunican. Una arquitectura adecuada facilita la mantenibilidad, seguridad, escalabilidad, rendimiento y evolución del producto de software.

En esta guía el aprendiz analizará los requerimientos del proyecto formativo y los transformará en decisiones arquitectónicas verificables. Se trabajará desde conceptos básicos de arquitectura y patrones, hasta la construcción de vistas, componentes, interfaces y despliegue. El propósito no es únicamente elaborar diagramas, sino justificar por qué una determinada estructura responde a las necesidades del proyecto.

La guía promueve el trabajo autónomo, colaborativo y sistemático. Cada actividad conduce a un producto que podrá integrarse al expediente técnico del proyecto de software. Se recomienda mantener una bitácora de decisiones arquitectónicas y versionar los artefactos en el repositorio del equipo.

3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

La secuencia de actividades se organiza de acuerdo con el formato GFPI-F-135 V04: reflexión inicial, contextualización e identificación de conocimientos, apropiación y transferencia del conocimiento. El desarrollo parte de situaciones del proyecto formativo y culmina con una propuesta arquitectónica sustentada.

3.1 Actividades de reflexión inicial

Descripción de la actividad:



Caso detonante: un equipo de desarrollo recibe el encargo de construir una aplicación para gestionar usuarios, productos, pedidos y pagos. El equipo comienza a programar directamente las funcionalidades, pero después de varias semanas aparecen código duplicado, dependencias difíciles de mantener, problemas de seguridad y dificultades para integrar nuevos servicios.

En equipos de tres aprendices, respondan y argumenten:

1. ¿Qué decisiones debieron tomarse antes de comenzar a programar?
2. ¿Qué consecuencias puede generar una arquitectura mal definida?
3. ¿Qué diferencia existe entre arquitectura, diseño detallado e implementación?
4. ¿Qué atributos de calidad deberían priorizarse en una aplicación de este tipo?
5. ¿Qué patrón o estilo arquitectónico considerarían inicialmente y por qué?

Producto: mapa conceptual o esquema argumentado sobre la importancia de la arquitectura de software.

Ambiente requerido: ambiente de formación con equipos de cómputo, conexión a Internet y tablero/proyector.

Estrategias o técnicas didácticas activas: estudio de caso, lluvia de ideas, discusión guiada y aprendizaje colaborativo.

Materiales de formación: computador, tablero, cuaderno o herramienta digital de diagramación.

Material de apoyo: documentación del proyecto formativo y lecturas sobre arquitectura de software.

Duración de la actividad: 4 horas.

3.2 Actividades de contextualización e identificación de conocimientos necesarios para el aprendizaje

Descripción de la actividad:

A partir del proyecto formativo, cada equipo realizará un diagnóstico arquitectónico inicial. Identificará actores, funcionalidades principales, restricciones técnicas, integraciones, datos relevantes y atributos de calidad. Posteriormente relacionará estos elementos con posibles estilos y patrones arquitectónicos.

Actividad diagnóstica:

1. Elabore una lista de los principales requerimientos funcionales y no funcionales del proyecto.
2. Clasifique los requerimientos no funcionales por atributos de calidad: seguridad, rendimiento, disponibilidad, mantenibilidad, escalabilidad y usabilidad.
3. Identifique los principales módulos o dominios funcionales.
4. Proponga dos alternativas de arquitectura y registre ventajas, desventajas y riesgos.
5. Seleccione preliminarmente una alternativa y formule una justificación técnica.

Producto: matriz de requisitos y decisiones arquitectónicas preliminares.

Ambiente requerido: aula de formación y/o ambiente virtual de aprendizaje.

Estrategias o técnicas didácticas activas: aprendizaje basado en proyectos, análisis comparativo y trabajo colaborativo.

Materiales de formación: historias de usuario/requisitos del proyecto, plantilla de matriz y herramienta de diagramación.

Material de apoyo: documentación técnica de patrones y estilos arquitectónicos.

Duración de la actividad: 6 horas.

3.3 Actividades de apropiación



Descripción de la actividad:

En esta fase el aprendiz construirá los conocimientos necesarios para diseñar una arquitectura de software y documentarla. Las actividades se desarrollarán de forma progresiva, aplicando cada concepto al proyecto formativo.

3.3.1 Conceptos fundamentales de arquitectura

Investigue, explique con sus propias palabras y ejemplifique:

- Arquitectura de software y arquitectura empresarial.
- Stakeholders y decisiones arquitectónicas.
- Requerimientos funcionales y no funcionales.
- Atributos de calidad y escenarios de calidad.
- Acoplamiento, cohesión, separación de responsabilidades y modularidad.
- Principios SOLID relacionados con el diseño.
- Deuda técnica y riesgos arquitectónicos.

Actividad práctica: elaborar una ficha técnica de atributos de calidad priorizados para el proyecto.

3.3.2 Estilos y patrones arquitectónicos

Analice y compare los siguientes enfoques: arquitectura en capas, cliente-servidor, MVC, monolito modular, microservicios, arquitectura hexagonal/puertos y adaptadores y Clean Architecture. Para cada uno registre estructura, ventajas, desventajas, contexto de aplicación y riesgos.

Actividad práctica: construir una matriz comparativa y seleccionar el estilo que mejor responda al proyecto.

Ver anexo: infografía estilos y patrones

3.3.3 Patrones de diseño

Investigue los patrones creacionales, estructurales y de comportamiento. Profundice especialmente en aquellos que puedan resolver necesidades reales del proyecto, por ejemplo Factory, Builder, Strategy, Observer, Adapter, Facade y Repository.

Actividad práctica: para tres problemas concretos del proyecto, proponga un patrón, explique el problema que resuelve y justifique su selección.

Ver anexo: Infografía Patrones de diseño

3.3.4 Vista de componentes

Defina los componentes principales del sistema, sus responsabilidades, interfaces y dependencias. El diagrama debe permitir identificar claramente qué componente consume o proporciona cada servicio.

Producto: diagrama de componentes UML acompañado de una tabla de responsabilidades e interfaces.

Ver anexo: Infografía vista de componentes

3.3.5 Vista de despliegue

Diseñe la distribución física o lógica de la solución: dispositivos, servidores, contenedores, servicios, bases de datos, redes y conexiones relevantes. Relacione el despliegue con requisitos de seguridad, disponibilidad y rendimiento.



Producto: diagrama de despliegue UML y descripción de cada nodo.

Ver anexo:Infografía vista de despliegue

3.3.6 Arquitectura de datos e integración

Defina la relación entre la arquitectura de aplicación y los mecanismos de persistencia e integración.

Considere API REST, autenticación/autorización, manejo de errores, transacciones, repositorios, mensajería cuando aplique y criterios de seguridad para la exposición de servicios.

Producto: esquema de integración y matriz de interfaces/API.

Ver anexo:Infografía datos e integración

3.3.7 Registro de decisiones arquitectónicas

Cada equipo elaborará un Architecture Decision Record (ADR) para las decisiones de mayor impacto. Como mínimo debe registrar contexto, decisión, alternativas consideradas, consecuencias y estado de la decisión.

Evidencias de aprendizaje

- Matriz de requisitos y atributos de calidad.
- Matriz comparativa de estilos arquitectónicos.
- Matriz de patrones de diseño seleccionados.
- Diagrama de componentes UML.
- Diagrama de despliegue UML.
- Esquema de integración y APIs.
- Mínimo tres ADR.
- Documento consolidado de arquitectura del proyecto.

Instrumentos de evaluación: lista de chequeo de arquitectura, rúbrica de diseño y sustentación técnica.

Duración de la actividad: 28 horas.

3.4 Actividades de Transferencia del Conocimiento

Descripción de la actividad:

Como producto final, cada equipo aplicará los conocimientos adquiridos a su proyecto formativo. Deberá entregar una propuesta de arquitectura técnicamente sustentada y coherente con los requerimientos del sistema.

1. Consolidar los requerimientos funcionales y no funcionales utilizados como base de las decisiones.
2. Definir el estilo o patrón arquitectónico seleccionado.
3. Elaborar la vista de contexto de la solución.
4. Elaborar la vista lógica y/o de contenedores según el nivel de detalle requerido.
5. Elaborar el diagrama de componentes.
6. Elaborar el diagrama de despliegue.
7. Documentar interfaces, dependencias y mecanismos de integración.
8. Documentar las decisiones arquitectónicas mediante ADR.
9. Identificar riesgos y estrategias de mitigación.
10. Sustentar la propuesta frente al instructor y responder preguntas técnicas.



Entregable final: Documento de Arquitectura de Software (DAS) en PDF y repositorio con los diagramas editables.

Ambiente requerido: ambiente de formación, repositorio Git y plataforma institucional de entrega.

Estrategias o técnicas didácticas activas: aprendizaje basado en proyectos, estudio de caso, revisión por pares, sustentación y demostración.

Materiales de formación: computador, repositorio Git, herramienta UML/diagramación, editor de texto y documentación del proyecto.

Material de apoyo: documentación técnica, guías de UML, patrones de diseño y referencias bibliográficas.

Evidencias de aprendizaje: documento de arquitectura, diagramas, ADR, matriz de trazabilidad y sustentación.

Instrumentos de evaluación: rúbrica de producto y lista de chequeo de sustentación.

Duración de la actividad: 10 horas.

3.5 Microservicios y tecnologías para el proyecto

Desarrollo taller Microservicios y dominios

Ver anexo PDF microservicios y tecnologías.

Ver anexo BP DDD

Ver anexo Consultas de gran escala

Ver anexo Optimizar tiempos de APPs

Ver anexo patrones Bd y microservicios

Ver anexo Clases inmutables

Ver anexo patrones BD

Ver anexo Bases de datos por dominio

Ver anexo diferencias arquitecturas

Ver anexo Dominios y patrones

Ver anexo Json y Mqtt

Ver anexo dominios

Ver Connection pool

Ver anexo autenticación



4. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN EN EL PROCESO FORMATIVO

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de aprendizaje	Evidencias de aprendizaje	Criterios de evaluación	Técnicas e instrumentos de evaluación
Planeación / Ejecución	Diseñar y documentar la arquitectura de la solución	Elaboración Matriz de requisitos y atributos de calidad	Matriz de requisitos y atributos de calidad	Relaciona requisitos con decisiones arquitectónicas y atributos de calidad.	Análisis de producto / Lista de chequeo
Planeación / Ejecución	Diseñar y documentar la arquitectura de la solución	Elaboración Matriz de estilos y patrones	Matriz de estilos y patrones	Selecciona estilos y patrones de acuerdo con el problema y justifica su elección.	Valoración de producto / Rúbrica
Planeación / Ejecución	Diseñar y documentar la arquitectura de la solución	Elaboración Diagrama de componentes y documentación	Diagrama de componentes y documentación	Representa componentes, responsabilidades, interfaces y dependencias de forma coherente.	Observación / Lista de chequeo
Planeación / Ejecución	Diseñar y documentar la arquitectura de la solución	Elaboración Diagrama de despliegue	Diagrama de despliegue	Relaciona nodos, servicios, infraestructura y comunicaciones con la solución propuesta.	Valoración de producto / Lista de chequeo
Planeación / Ejecución	Diseñar y documentar la arquitectura de la solución	Elaboración Documento de Arquitectura de Software + ADR + sustentación	Documento de Arquitectura de Software + ADR + sustentación	Desarrolla la arquitectura de acuerdo con el patrón seleccionado y sustenta técnicamente las decisiones.	Proyecto / Rúbrica y sustentación

5. GLOSARIO DE TÉRMINOS

ADR: Architecture Decision Record. Registro de una decisión arquitectónica, su contexto, alternativas y consecuencias.

API: Interfaz que permite la comunicación entre componentes o sistemas mediante contratos definidos.

Arquitectura de software: Estructura fundamental de un sistema, sus componentes, relaciones y principios que orientan su evolución.

Atributo de calidad: Característica no funcional que permite valorar propiedades como seguridad, rendimiento o mantenibilidad.

Acoplamiento: Grado de dependencia entre módulos o componentes.

Cohesión: Grado en que las responsabilidades de un módulo están relacionadas entre sí.

Componente: Unidad modular de software con responsabilidades e interfaces definidas.

Despliegue: Distribución de componentes de software sobre nodos o infraestructura.

DDD: Domain-Driven Design; enfoque que centra el diseño en el dominio y sus reglas de negocio.

Escalabilidad: Capacidad de una solución para atender incrementos de carga manteniendo niveles aceptables de servicio.



Interfaz: Contrato mediante el cual un componente expone o consume operaciones o servicios.

Microservicio: Servicio autónomo orientado a una capacidad de negocio y comunicable mediante interfaces definidas.

Patrón de diseño: Solución reutilizable a un problema recurrente de diseño de software dentro de un contexto.

Patrón arquitectónico: Estructura de alto nivel para organizar una solución y resolver problemas arquitectónicos recurrentes.

Repositorio: Componente o mecanismo que abstrae operaciones de persistencia y acceso a datos.

UML: Lenguaje Unificado de Modelado utilizado para representar diferentes vistas y aspectos de un sistema.

6. REFERENTES BIBLIOGRÁFICOS

- Bass, L., Clements, P., & Kazman, R. Software Architecture in Practice. Addison-Wesley.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Pearson.
- Richards, M., & Ford, N. Fundamentals of Software Architecture: An Engineering Approach. O'Reilly Media.
- Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley.
- Object Management Group. Unified Modeling Language (UML), especificaciones y recursos oficiales.
- Documentación técnica oficial de las tecnologías utilizadas en el proyecto formativo.
- Formato GFPI-F-135 V04, utilizado como plantilla institucional para esta guía.

7. CONTROL DEL DOCUMENTO

Autor(es)	Nombre	Cargo	Dependencia	Fecha
Autor (es)	Javier Humberto Pinto	Instructor	Centro de Formación CIES	2026-08-01

8. CONTROL DE CAMBIOS

Autor(es)	Nombre	Cargo	Dependencia	Fecha	Razón del Cambio
Autor (es)	Javier Humberto Pinto	Instructor	Centro de Formación CIES	2026-08-26	Diseño de guía de aprendizaje para arquitectura de software, tomando como base microservicios